Data Structures and Algorithms compact cheatsheet

STEP 1 Getting started

1.1 Introduction to DSA

What data structures and algorithms are A data structure is a way of organising data in memory so that the operations you need are easy and fast. A list of numbers kept in order, a stack of undo steps and a map from names to phone numbers are all data structures.

Why DSA matters Computers are fast, roughly a hundred million to a billion simple steps per second. That speed hides bad choices on small inputs and exposes them on large ones.

What makes something an algorithm Donald Knuth listed five properties that every algorithm must have.

Kinds of data structures Both hold items in a line. They differ only in which end you remove from, and that one difference makes them fit completely different jobs.

Abstract data types An abstract data type (ADT) describes what a type does: its values and its operations, with the rules they follow. It says nothing about how those operations are built. A data structure is one concrete way to build an ADT.

Core operations and what they cost Almost every task is a mix of a few basic operations: access (read by position), search (find by value), insert , delete , traverse (visit everything), sort and merge . Each structure makes some of these cheap and others expensive.

Algorithm design paradigms Most algorithms are built from a small set of strategies. Recognising which one fits is half of solving a problem.

Choosing the right structure

A method for solving any problem

EXAM Technical interviews test DSA because it shows whether you can reason about correctness and cost, not just make code run.

TRAP In JavaScript, `array.shift()` removes the first element by moving every other one, which is O(n). The example keeps an index to the front instead, so each removal is O(1).

TRAP With coins 1, 3 and 4, greedy takes 4 first and then needs two 1s. Dynamic programming discovers that 3 + 3 uses only two coins. A greedy idea must be proved before it is trusted.

1.2 Run time analysis

Why count steps instead of timing Timing a program with a stopwatch tells you about one machine, one language, one input and one moment. Change any of these and the number changes. We want a measure that belongs to the algorithm itself .

Orders of growth From best to worst: $1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$

Big O, Big Omega and Big Theta Asymptotic notation compares how two functions grow when n becomes large. Here f(n) is the step count and g(n) is a simple reference such as n^2.

Rules for finding the complexity The triangle loop does 1 + 2 + ... + (n − 1) = n(n − 1) 2 = 64 × 63 2 = 2016 steps. Half of n^2 is still $O(n^2)$.

Best, worst, average and amortized Some operations are usually cheap and occasionally expensive. Amortized analysis spreads the rare expensive cost over the whole sequence.

Space complexity Space complexity measures how the memory an algorithm uses grows with n. It has two parts:

Auxiliary space in depth When someone asks for the space complexity of an algorithm, they almost always mean its auxiliary space . The input has to be stored anyway, so including it would make every algorithm at least O(n) and hide the real differences.

Recurrences and the recursion tree A recursive function's cost is described by a recurrence : the cost for n written in terms of the cost for smaller inputs. For merge sort:

Recursion depth and the call stack Every function call pushes a stack frame holding its parameters, local variables and the place to return to. The frame is popped when the call returns.

From constraints to complexity A rough rule: a typical judge or server does about 10^8 simple steps per second. The input limits then tell you which complexity is acceptable for a one second limit.

Why count steps instead of timing

$$T(n) = n + 2$$

Orders of growth

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$$

Big O, Big Omega and Big Theta

$$3n^2 + 5n + 2 = O(n^2)$$

Rules for finding the complexity

$$1 + 2 + \ldots + (n - 1) = \frac{n(n-1)}{2} = \frac{64 \times 63}{2} = 2016$$

Best, worst, average and amortized

$$1 + 2 + 4 + \ldots < 2n$$

Space complexity

$$\text{Total space} = \text{input space} + \text{auxiliary space}$$

Auxiliary space in depth $\log_2 16 + 1 = 5$

Recurrences and the recursion tree

$$T(n) = 2T(n/2) + n,\ T(1) = 1$$

TRAP A COMMON MIX-UP. Big O is not the same thing as "worst case". Big O is an upper bound on a function. The worst case running time, the best case running time and the average running time are three different functions, and each one can be described with O, Ω or Θ.

EXAM If an algorithm is O(n), it is also $O(n^2)$ and $O(2^n)$, since those are still upper bounds. When people say "the complexity is O(n)" they mean the tightest bound, which is really Θ(n).

TRAP A recursive factorial or list walk on a million items will crash with a stack overflow, even though its time is fine. Convert deep linear recursion into a loop, which needs O(1) stack space.

STEP 2 Core building blocks

2.1 Mathematics in DSA

Primality Testing √N bound with 6k ± 1 candidates: O(√N), against O(N) for try every d from 2 to n − 1.

GCD and LCM (Euclidean Algorithm) Euclidean algorithm: O(log min(a, b)), against O(min(a, b)) for count down from min(a, b).

Counting Digits and Palindrome Number Arithmetic with half reversal: O(d), against O(d) for convert to a string.

All Divisors of N Pairs up to √n: O(√n), against O(n) for test every d from 1 to n.

Power of Two and Counting Set Bits Kernighan and n & (n − 1): O(set bits) / O(1), against O(32) / O(log n) for scan all 32 bits / divide by 2.

Base Conversion (Any Base 2 to 36) Repeated division and Horner: O(log_b n), against O(b · log_b n) for greedy subtraction of powers.

Nth Fibonacci Number Iterate with two variables: O(n), against O(φⁿ) ≈ O(1.618ⁿ) for plain recursion.

Sieve of Eratosthenes Sieve of Eratosthenes: O(N log log N), against O(N √N) for trial-divide each number.

Modular Exponentiation (Fast Power) Binary exponentiation: O(log exp), against O(exp) for repeated multiplication.

Prime Factorization Trial division to √n: O(√n), against O(n) for prime n for divide by every d until n = 1.

Binomial Coefficient nCr mod p Factorials and Fermat inverses: O(n + log p) build, O(1) query.

Integer Square Root Binary search on the answer: O(log n), against O(√n) for count up.

Inclusion-Exclusion: Count Multiples Inclusion-exclusion over subsets: O(2^k · k log N), against O(N · k) for check every x in 1 ... n.

Modular Inverse (Extended Euclid) Extended Euclidean algorithm: O(log m), against O(m) for try every x.

Matrix Exponentiation (Nth Fibonacci in O(log N)) Matrix binary exponentiation: O(log n) (8 mults per step), against O(n) for iterate n steps.

Catalan Numbers Ratio from the closed form: O(n) big-number steps, against about O(3ⁿ) for plain recursion.

Chinese Remainder Theorem Pairwise merging with Extended Euclid: O(k log M), against O(lcm · k) for search 0 ... lcm − 1.

Segmented Sieve (Primes in a Range) Segmented sieve: O((R − L) log log R + √R), against O((R − L) √R) for trial-divide each number in [l, r].

Miller-Rabin Primality Test Deterministic Miller-Rabin: O(12 · log³ n), against O(√n) for trial division to √n.

Primality Testing

$$N = a \times b \Rightarrow \min(a, b) \le \sqrt{N}$$

GCD and LCM (Euclidean Algorithm)

$$\gcd(a, b) = \gcd(b, \mathbf{a \bmod b}),\quad \gcd(a, 0) = |a|$$

Counting Digits and Palindrome Number
digit = n mod 10, n = ⌊n / 10⌋

All Divisors of N
$i \mid n \iff (n / i) \mid n, \quad \min(i, n / i) \le \sqrt{n}$

Power of Two and Counting Set Bits
x & (x − 1) clears the **lowest set bit**

Base Conversion (Any Base 2 to 36)
$n = d_k b^k + \ldots + d_1 b + d_0$

Nth Fibonacci Number
F(n) = F(n − 1) + F(n − 2)

Sieve of Eratosthenes
for p = 2 … √N: if p unmarked, mark **p^2, p^2 + p, …**

Modular Exponentiation (Fast Power)
$a^e = (a^2)^{e/2}$ if e is even, $a \cdot a^{e-1}$ if e is odd

Factorial and Trailing Zeros
$\text{zeros}(n!) = \min(v_2, v_5) = v_5(n!)$

Prime Factorization $n = p_1^{e_1} \cdot p_2^{e_2} \cdot \ldots \cdot p_k^{e_k}$

$\varphi(n) = n \cdot \prod_{p \mid n} (1 - 1/p)$

Binomial Coefficient nCr mod p
C(n, r) = C(n−1, r−1) + C(n−1, r)

Integer Square Root $x = \max\{ k : k^2 \le n \}$

Inclusion-Exclusion: Count Multiples
$|A_1 \cup \ldots \cup A_k| = \Sigma_{S \neq \varnothing} (-1)^{|S|+1} \lfloor N / \text{lcm}(S) \rfloor$

Modular Inverse (Extended Euclid)
a · x + m · y = gcd(a, m) (Bezout)

Matrix Exponentiation (Nth Fibonacci in O(log N))
$[[1, 1], [1, 0]]^n = [[F(n+1), \mathbf{F(n)}], [F(n), F(n-1)]]$

$n = \Sigma\, n_i\, p^i, \quad r = \Sigma\, r_i\, p^i$

Catalan Numbers
$C(0) = 1, \quad C(n) = \Sigma_{i=0}^{n-1} C(i) \cdot C(n - 1 - i)$

Chinese Remainder Theorem
$x = \text{cur} + M \cdot t, \quad M \cdot t \equiv a - \text{cur} \pmod m$

Segmented Sieve (Primes in a Range)
start(p) = max(p^2, ⌈L / p⌉ · p)

Miller-Rabin Primality Test
$n - 1 = d \cdot 2^s, \quad d \text{ odd}$

2.2 Arrays

Memory Allocation: Stack vs Heap Stack allocation: memory is reserved by moving the stack pointer. It is very fast and is released automatically when the function returns. Size must be small and is usually known in advance (C: `int a[5]`).

Contiguous Memory Layout An array is one unbroken block of bytes. Element `i` starts at byte offset `i × size`.

Cache Locality Spatial locality: if you touch address *x*, you will probably touch *x+1* soon. Hardware loads whole cache lines (usually 64 bytes, or 16 ints).

Array Indexing Mechanics 0-based: C, C++, Java, JavaScript, Python, Go, Rust. The index equals the offset in elements, so no extra subtraction is needed.

Static Arrays Compile-time size: the length is a constant in the source code (C: `int a[10]`). The compiler can reserve it on the stack or in static storage.

Dynamic Arrays and Amortized Cost Keeps **size** (used slots) and **capacity** (allocated slots). Push writes at `size` when `size < capacity`.

Insertion, Deletion and Shifting Insert at i: shift `a[i..n−1]` right by one, starting from the end so nothing is overwritten. Cost O(n − i).

Multi-Dimensional Arrays Row-major: C, C++, NumPy default (`order='C'`). **Column-major**: Fortran, MATLAB, R, Julia, NumPy `order='F'`.

Jagged Arrays Also called an **array of arrays** or **array of pointers**. The outer array stores references, each row is a separate heap block.

Linear Search Works on **unsorted** data. No setup needed.

Binary Search Needs a **sorted** array. Time O(log n), space O(1) when written as a loop.

Bubble, Insertion and Selection Sort All three are **in-place** (O(1) extra space) and $O(n^2)$ in the worst case.

Merge Sort and Quick Sort Merge sort: O(n log n) always, stable, needs O(n) extra space for the temporary array. Great for linked lists and external sorting.

Heap Sort A **binary heap** is stored in the array itself: the children of index i are

$$2i + 1$$

and

$$2i + 2$$

, its parent is

$$\lfloor (i - 1) / 2 \rfloor$$

.

TimSort and Introsort (Hybrid Sorts) TimSort (Python `sorted`, Java `Arrays.sort` for objects, V8 `Array.prototype.sort`): find natural runs, reverse strictly descending ones, extend short runs to **minrun** (32 to 64 in practice) with binary insertion sort, then merge runs kept on a stack with balance rules. Merges use **galloping** when one run keeps winning.

Shell, Comb, Cocktail, Gnome and Odd-Even Sort Shell sort: insertion sort on elements a gap apart, with gaps shrinking to 1. Knuth gaps 1, 4, 13, 40 ... give about $O(n^{1.5})$. In place, not stable, very little code.

Counting, Radix and Bucket Sort Comparison sorts need at least

$$\log_2(n!) \approx n \log_2 n$$

comparisons. These sorts avoid the bound by using the values themselves as positions.

Cycle Sort and Pancake Sort Cycle sort makes the minimum possible number of writes: every value is written at most once into its final position. Time $O(n^2)$, in place, not stable. Useful when writes are expensive, such as flash memory.

Choosing a Sorting Algorithm Small or nearly sorted: insertion sort, which is why TimSort and introsort use it for small pieces.

Two Pointers Technique Opposite direction (converging): `l = 0, r = n − 1`, move one or both inward each step. O(n) time, O(1) space.

Sliding Window Fixed window of size k: keep a running sum. `sum += a[r] − a[r − k]`. Total O(n) instead of O(n·k).

Prefix Sums and Difference Arrays Prefix array of length n + 1 with `P[0] = 0` and `P[i+1] = P[i] + a[i]`. Build O(n), each range-sum query O(1).

Frequency Arrays When keys are small integers in a known range [0, K), an array of size K is a perfect hash table with no collisions.

A dynamic programming idea: `current` is the best sum of a subarray that **ends exactly at i**.

Dutch National Flag Edsger Dijkstra's three-way partition. One pass, O(n) time, O(1) space, in place.

Boyer-Moore Majority Vote Majority (> ⌊n/2⌋): one candidate and one counter. O(n) time, O(1) space.

Matrix Manipulations Rotate 90° clockwise in place: transpose (`swap a[i][j], a[j][i]` for j > i), then reverse every row. $O(n^2)$ time, O(1) space.

Monotonic Stack and Queue A **monotonic stack** keeps values strictly increasing or decreasing from bottom to top. Each index is pushed once and popped once, so O(n) total.

Two-Dimensional Prefix Sum Build: `P[i+1][j+1] = a[i][j] + P[i][j+1] + P[i+1][j] − P[i][j]`. O(rows × cols) time and space.

Interval Overlapping Patterns Merge overlapping: sort by start (O(n log n)), then one pass. If `start ≤ last.end`, set `last.end = max(last.end, end)`, else append.

Square Root Decomposition and Mo's Algorithm Block size `b = ⌈√n⌉`. Precompute `block[k]` = sum of its b elements in O(n).

Fenwick Trees and Segment Trees Both support **point update** and **range query** in **O(log n)**. A plain array is O(1) and O(n), a prefix array is O(n) and O(1).

2.3 Strings

What a string really is A string is a sequence of characters: a name, a sentence, a DNA sequence, a line of a log file. Underneath, the computer does not store letters at all. It stores numbers , one or more per character, and an encoding says which number means which character.

Immutable strings and building them fast In Python, JavaScript and Java a string is immutable : once made, it never changes. Methods such as upper() or replace() return a new string and leave the old one alone.

Everyday operations and what they cost Every language gives you a toolbox for strings. Knowing what each tool costs stops you from hiding an O(n) step inside a loop.

Two pointers: reverse and palindromes Many string problems become easy with two indexes that walk toward each other: i from the left and j from the right. Each step moves at least one of them, so the whole walk is O(n) with O(1) extra space.

Counting characters: anagrams and frequency Two words are anagrams when they use exactly the same letters the same number of times, like "listen" and "silent". Sorting both words and comparing works in O(n log n). Counting is faster.

Sliding window: the longest run without repeats A window is a stretch of the string between two indexes, start and end . Instead of checking every one of the $n^2/2$ substrings, grow the window on the right one character at a time and shrink it on the left only when a rule is broken.

Pattern matching: naive search and KMP Finding a pattern of length m inside a text of length n is one of the oldest string problems. Your editor's find box, grep and virus scanners all do it.

Rolling hash: Rabin and Karp Another way to search is to give every window of the text a fingerprint , a number computed from its characters, and compare fingerprints instead of characters.

Immutable strings and building them fast
$1 + 2 + 3 + \ldots + n = n(n + 1) / 2 \approx n^2 / 2$

Counting characters: anagrams and frequency
index of a letter = code(letter) − code('a') for example code('c') − 97 = 99 − 97 = 2

Pattern matching: naive search and KMP
lps[i] = length of the longest proper prefix of p[0..i] that is also a suffix of p[0..i]

Rolling hash: Rabin and Karp
hash(window) = (c_0·B^(m−1) + c_1·B^(m−2) + … + c_(m−1)) mod M

TRAP An emoji such as 😀 is one character to you, but two UTF-16 units in Java and JavaScript, so `"😀".length` is 2 there and 1 in Python. Be careful when you count characters from user input.

EXAM In Java, == compares whether two variables point at the same object. Use `a.equals(b)` to compare the characters. In Python and JavaScript, == and === compare the characters.

STEP 3 Linear structures

3.1 Linked lists

Nodes holding hands A linked list is a chain of nodes . Each node holds a value and a reference (a pointer) to the next node. The list itself only remembers the first node, called the head . The last node points at nothing: None in Python, null in JavaScript and Java.

Inserting and deleting by rewiring Nothing ever moves in a linked list. Inserting or deleting only changes a few references, which is why these operations can be O(1) once you are standing at the right place.

Doubly linked and circular lists A doubly linked list gives every node two references: next and prev . It costs one more reference per node, and in return you can walk backwards and remove a node in O(1) when you hold that node, because it knows both of its neighbours.

Reversing a list in place Reversing is the classic linked list exercise, and it trains the most important habit: save the next node before you change a reference .

Fast and slow pointers Send two pointers down the list at different speeds: the tortoise moves one node per step and the hare moves two. This one idea answers several questions in a single pass with O(1) extra space.

Merging two sorted lists Given two lists that are already sorted, build one sorted list by repeatedly taking the smaller front node. No new nodes are needed: we only rewire the existing ones. A dummy node again saves us from special cases at the start.

Lists versus arrays in real code On paper, linked lists win at inserting and deleting. In real programs, arrays often win anyway, because their boxes sit next to each other and the CPU cache loads many of them at once. Following references jumps around memory.

Fast and slow pointers
gap after each step = gap − 1 so they meet in at most (loop length) steps

TRAP Order matters. When inserting after a node, set `new.next` first. If you set `node.next = new` first, the rest of the list is lost.

EXAM To find where the loop STARTS , put one pointer back at the head after they meet and move both one step at a time. They meet again exactly at the start of the loop.

3.2 Stacks and queues

The stack: last in, first out A stack is a pile. You add to the top (push), look at the top (peek) and remove from the top (pop). The last item in is the first item out, which is called LIFO .

Matching brackets with a stack Code editors, compilers and HTML checkers all need to know whether brackets are properly nested. A stack does it in one pass: push every opener, and when a closer arrives, the top of the stack must be its partner.

Evaluating expressions People write infix expressions like (3 + 4) * 2 − 5 , with operators between the numbers and parentheses for order. Computers prefer postfix (also called reverse Polish notation), where each operator comes after its two operands: 3 4 + 2 * 5 − .

The queue: first in, first out A queue is a line at a ticket counter. Items join at the back (enqueue) and leave from the front (dequeue). The first item in is the first item out: FIFO .

Deques and priority queues A heap is a complete binary tree stored in an array: the children of index i sit at 2i + 1 and 2i + 2, and every parent is smaller than its children. The Trees topic explains it in detail.

Monotonic stacks A monotonic stack keeps its items in increasing or decreasing order. It answers "what is the next bigger (or smaller) item to my right?" for every position in a single pass.

Undo, redo and other everyday uses Undo and redo are two stacks. Every change pushes the old state onto the undo stack. Undo moves the current state to the redo stack and pops the previous one. A brand new change clears the redo stack, because the future it remembered no longer exists.

The queue: first in, first out
tail = (head + size) mod capacity head = (head + 1) mod capacity

Monotonic stacks pushes + pops ≤ n + n = 2n

TRAP In Java, avoid the old `Stack` class. It is synchronized and slower. `ArrayDeque` is the recommended stack and queue.

EXAM Pop order matters for − and /. The first pop gives the RIGHT operand. For 8 2 − you pop 2, then 8, and compute 8 − 2 = 6, not 2 − 8.

TRAP Removing the first element of a Python list (`pop(0)`) or a JavaScript array (`shift()`) moves every other element, which is O(N) . Use `collections.deque` in Python, `ArrayDeque` in Java, and a head index or a ring buffer in JavaScript.

STEP 4 Trees and graphs

4.1 Trees

What a tree is A tree stores items in a hierarchy. One node at the top is the root . Every other node has exactly one parent , and any node can have children below it. A node with no children is a leaf .

Traversals: visiting every node A traversal visits every node once. Depth first traversals go deep before going wide, and differ only in when the node itself is visited compared with its two subtrees.

Binary search trees A binary search tree (BST) adds one rule: everything in the left subtree of a node is smaller than the node, and everything in the right subtree is larger. That rule turns every node into a signpost.

Deleting from a search tree Deleting is the only tricky BST operation, because the tree must keep its ordering rule. First find the node, then handle one of three cases.

Balance: why shape matters A BST is only fast when it is short . Insert values that are already sorted and every new node becomes a right child, so the tree turns into a chain that is no better than a linked list.

Heaps: trees inside arrays A binary heap is a complete binary tree: every level is full except possibly the last, which fills from the left. In a min heap every parent is smaller than or equal to its children, so the smallest value is always at the root.

Tries: trees of letters A trie (also called a prefix tree) stores words letter by letter. Each edge is one letter, and words that share a beginning share the same path. A flag on a node marks where a complete word ends.

🔢 **Edges** edges = n − 1

Balance: why shape matters $h = \lceil \log_2(n + 1) \rceil$

Heaps: trees inside arrays
left = 2i + 1 right = 2i + 2 parent = ⌊(i − 1) / 2⌋

EXAM Inorder traversal of a BST always produces the values in sorted order. Exams love asking for it, and it is also the quickest way to check that a tree really is a BST.

TRAP Python and JavaScript have no built in balanced search tree. For sorted keys in Python, the `bisect` module on a sorted list or the third party `sortedcontainers` package is the usual answer.

4.2 Graphs

What a graph is A graph is a set of vertices (also called nodes) joined by edges . Unlike a tree, a graph has no root and may contain loops, so you can walk in a circle and come back to where you started.

Breadth first search Breadth first search (BFS) explores a graph in rings. It visits the start, then every vertex one edge away, then every vertex two edges away, and so on. A queue keeps the discovered vertices in order, and a seen mark stops any vertex from being added twice.

Depth first search Depth first search (DFS) follows one path as far as it can go, then backs up to the last vertex with an unexplored neighbour and tries again. Recursion gives it a stack for free. Switch the animation above to DFS to compare the two orders.

Topological order Some tasks must happen before others: socks before shoes, a course before the next course, a library before the program that uses it. Draw an arrow from each task to the task that depends on it, and you get a directed acyclic graph (DAG).

Dijkstra's shortest paths When edges have weights, the path with the fewest edges is not always the cheapest. Dijkstra's algorithm grows a set of settled vertices whose shortest distance is final.

Minimum spanning trees and union find A town wants to connect every house to the network with the least cable. A spanning tree connects all V vertices with exactly V – 1 edges and no cycle. The minimum spanning tree (MST) is the one with the smallest total weight.

Grids are graphs too Many puzzles hide a graph inside a grid. Every cell is a vertex, and it is joined to its four neighbours up, down, left and right. You never build an adjacency list. You compute the neighbours with small offsets and check that they stay inside the grid.

🤝 **Handshakes** sum of all degrees = 2E

Breadth first search O(V + E)

Dijkstra's shortest paths
if dist[v] + w < dist[u] then dist[u] = dist[v] + w

TRAP Deep recursion can overflow the call stack on a graph with hundreds of thousands of vertices in a line. Python stops at about 1000 levels by default. For large inputs, write DFS with your own stack.

EXAM If the order ends up shorter than the number of vertices, some vertices never became ready. That only happens when the graph has a CYCLE , so Kahn's algorithm is also a cycle detector.

TRAP Dijkstra needs weights that are zero or positive. A negative edge can make a settled distance wrong. For negative weights use Bellman-Ford, which relaxes every edge V – 1 times in O(V · E).